```
def partition[S](arr: STArray[S,Int],
                 n: Int, r: Int, pivot: Int): ST[S,Int]
def qs[S](a: STArray[S,Int], n: Int, r: Int): ST[S,Unit]
```

With those components written, `quicksort` can now be assembled out of them in the `ST` monad:

```
def quicksort(xs: List[Int]): List[Int] =
  if (xs.isEmpty) xs else ST.runST(new RunnableST[List[Int]] {
    def apply[S] = for {
      arr    <- STArray.fromList(xs)
      size   <- arr.size
      _      <- qs(arr, 0, size - 1)
      sorted <- arr.freeze
    } yield sorted
  })
```

As you can see, the `ST` monad allows us to write pure functions that nevertheless mutate the data they receive. Scala's type system ensures that we don't combine things in an unsafe way.

EXERCISE 14.3

Give the same treatment to `scala.collection.mutable.HashMap` as we've given here to references and arrays. Come up with a minimal set of primitive combinators for creating and manipulating hash maps.

14.3 Purity is contextual

In the preceding section, we talked about effects that aren't observable because they're entirely local to some scope. A program can't observe mutation of data unless it holds a reference to that data.

But there are other effects that may be non-observable, depending on who's looking. As a simple example, let's take a kind of side effect that occurs all the time in ordinary Scala programs, even ones that we'd usually consider purely functional:

```
scala> case class Foo(s: String)

scala> val b = Foo("hello") == Foo("hello")
b: Boolean = true

scala> val c = Foo("hello") eq Foo("hello")
c: Boolean = false
```

Here `Foo("hello")` looks pretty innocent. We could be forgiven if we assumed that it was a completely referentially transparent expression. But each time it appears, it produces a *different* `Foo` in a certain sense. If we test two appearances of `Foo("hello")` for equality using the `==` function, we get `true` as we'd expect. But testing for *reference equality* (a notion inherited from the Java language) with `eq`, we get `false`. The two appearances of `Foo("hello")` aren't references to the "same object" if we look under the hood.

Note that if we evaluate `Foo("hello")` and store the result as `x`, then substitute `x` to get the expression `x eq x`, it has a different result:

```
scala> val x = Foo("hello")
x: Foo = Foo(hello)

scala> val d = x eq x
d: Boolean = true
```

Therefore, by our original definition of referential transparency, *every data constructor in Scala has a side effect*. The effect is that a new and unique object is created in memory, and the data constructor returns a reference to that new object.

For most programs, this makes no difference, because most programs don't check for reference equality. It's only the `eq` method that allows our programs to observe this side effect occurring. We could therefore say that it's not a side effect at all in the context of the vast majority of programs.

Our definition of referential transparency doesn't take this into account. Referential transparency is *with regard to* some context and our definition doesn't establish this context.

> **A more general definition of referential transparency**
> An expression `e` is referentially transparent with regard to a program `p` if every occurrence of `e` in `p` can be replaced by the result of evaluating `e` without affecting the meaning of `p`.

This definition is only slightly modified to reflect the fact that not all programs observe the same effects. We say that an effect of `e` is *non-observable* by `p` if it doesn't affect the referential transparency of `e` with regard to `p`. For instance, most programs can't observe the side effect of calling a constructor, because they don't make use of `eq`.

This definition is still somewhat vague. What is meant by "evaluating"? And what's the standard by which we determine whether the meaning of two programs is the same?

In Scala, there's a kind of standard answer to this first question. We'll take evaluation to mean *reduction to some normal form*. Since Scala is a strictly evaluated language, we can force the evaluation of an expression `e` to normal form in Scala by assigning it to a `val`:

```
val v = e
```

And referential transparency of `e` with regard to a program `p` means that we can rewrite `p`, replacing every appearance of `e` with `v` without changing the meaning of our program.